

Lab 3: Programming a Real Processing-in-Memory Architecture

Introduction

In this lab, I performed an analysis of a real Processing-in-Memory (PIM) architecture. This involved implementing and evaluating four distinct tasks for the UPMEM PIM System: characterizing data transfer performance, scaling the AXPY vector operation, assessing performance across different data types and operations, and exploring intra-DPU synchronization primitives for parallel vector reduction. Each task was simulated using the simulator provided by the UPMEM SDK.

Task 1 Transferring Data between Main Memory and PIM-enabled Memory

This task was concerned with the data transfer bandwidth between the host CPU and the DPUs. Aside from computation inputs and outputs being transferred between the host CPU and the DPUs, communication between DPUs has also to be routed through the host as an intermediary. Therefore having high transfer bandwidth is important for the PIM system to be effective. Documentation for runtime communication primitives is available in the UPMEM manual.

Implementation Details

There are three different data transfer methods: **SERIAL**, **PARALLEL** and **BROADCAST**. Data transfers have to be invoked on the host, regardless of the transfer direction (CPU to DPU or DPU to CPU). In contrast to the former two methods, **BROADCAST** can only distribute data from the host to the DPUs. The **SERIAL** and **PARALLEL** methods rely on the `dpu_prepare_xfer` function. The **BROADCAST** method relies on the `dpu_broadcast_to` function. In my implementation, the data is written to and read from each DPU's `DPU_MRAM_HEAP_POINTER`. The MRAM capacity of each DPU limits the theoretical maximum transfer size per DPU to 64MB. Each transfer size has to be aligned to 8 bytes.

SERIAL **SERIAL** passes the `DPU_XFER_DEFAULT` option, which indicates that the transfer should happen synchronously. Although the transfer between the host and the dpus occurs in parallel, each call to `dpu_push_xfer` is blocking. Later calls to `dpu_push_xfer` can only start once the first call has completed. Note there can an alternative way of implementing serial communication, depending on how the task description is understood. This involves iterating over the dpus individually and transferring the data one-by-one. Both interpretations are reasonable, so I implemented the first one based on the fact that it was less effort to program and intuitively performed better.

PARALLEL **PARALLEL** passes the `DPU_XFER_ASYNC` option, requesting an asynchronous transfer. This also starts a parallel transfer, but it allows for subsequent transfers to start before every single parallel transfer of the first call has completed. To ensure data integrity in later computation, `dpu_sync` has to be called after a parallel transfer to wait for every asynchronous operation to have completed.

BROADCAST **BROADCAST** calls `dpu_broadcast_to`, which copies the same buffer to all DPUS in the set.

```
#ifdef SERIAL    // Serial transfers
    DPU_ASSERT(dpu_push_xfer(
        dpu_set,                // the identifier of the DPU set
        DPU_XFER_TO_DPU,        // direction of the transfer
        DPU_MRAM_HEAP_POINTER_NAME, // location of symbol where to place data
        0,                      // byte offset from symbol address
        input_size_dpu_8bytes * sizeof(T), // number of bytes to copy
        DPU_XFER_DEFAULT        // options of the transfer
    ));
#elif BROADCAST // Broadcast transfers
    DPU_ASSERT(dpu_broadcast_to(
        dpu_set,
        DPU_MRAM_HEAP_POINTER_NAME,
        0,
        bufferX,
```

```

        input_size_dpu_8bytes * sizeof(T),
        DPU_XFER_DEFAULT
    ));

    #else // Parallel transfers
        DPU_ASSERT(dpu_push_xfer(
            dpu_set,
            DPU_XFER_TO_DPU,
            DPU_MRAM_HEAP_POINTER_NAME,
            0,
            input_size_dpu_8bytes * sizeof(T),
            DPU_XFER_ASYNC
        ));

        // wait for every asynchronous operation to have been performed
        DPU_ASSERT(dpu_sync(dpu_set));
    #endif

```

Evaluation

To evaluate the effectiveness of each transfer method, I transferred data between the host and between 1 and 64 DPUs at three different transfer sizes. The final result per DPU count was an average gained from ten separate runs. The transfer bandwidth is calculated as the total size of the transfer, divided by transfer duration.

Distributing data from the CPU to DPUs As can be seen in figure fig. 1, the broadcast method consistently provides the highest bandwidth in the CPU to DPU transfers, while the serial and parallel methods perform similarly overall. The performance benefit between broadcasting and the other two methods is more pronounced for larger transfer sizes. At transfer sizes of 24MB and 48MB, the measured bandwidth increases sharply till around 8 DPUs and then drops off linearly thereafter at a much slower rate.

Collecting data from the DPUs In this scenario, the highest bandwidth is measured at DPU counts less than 5. Thereafter, the bandwidth remains at an almost constant level for the remaining DPU counts. There is very little difference between the serial and parallel transfer methods.

Analysis and Observations

- CPU→DPU is consistently faster than DPU→CPU.
- SERIAL and PARALLEL look similar here because the asynchrony only helps when you queue multiple outstanding transfers before waiting. In our implementation we issue one transfer and then immediately call `dpu_sync`, which collapses the overlap and makes it behave close to the synchronous version. To expose a benefit, we would need to pipeline several transfers (e.g., multiple chunks or directions) before the sync.
- BROADCAST delivers the highest CPU→DPU bandwidth, indicating that fixed overheads are better amortized compared to the other transfer methods.

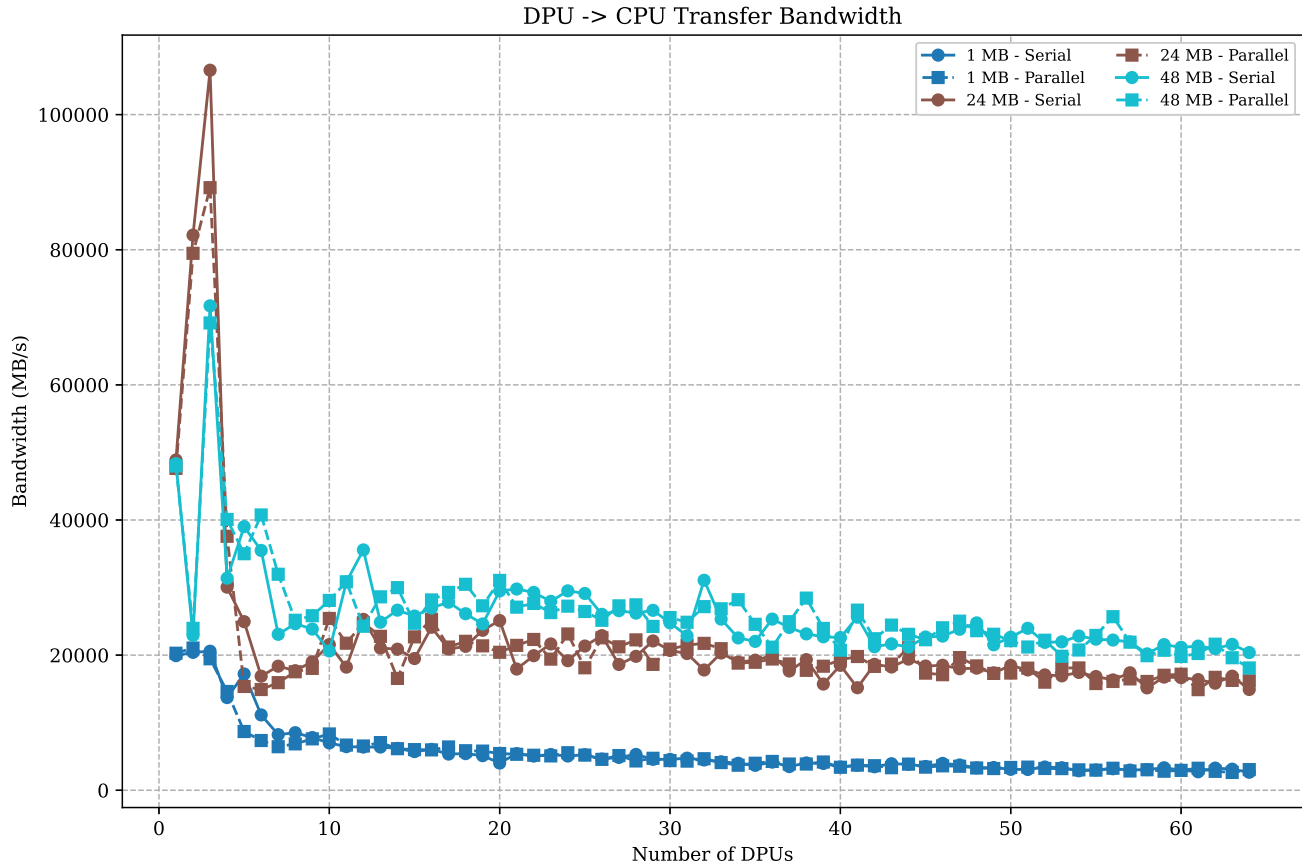
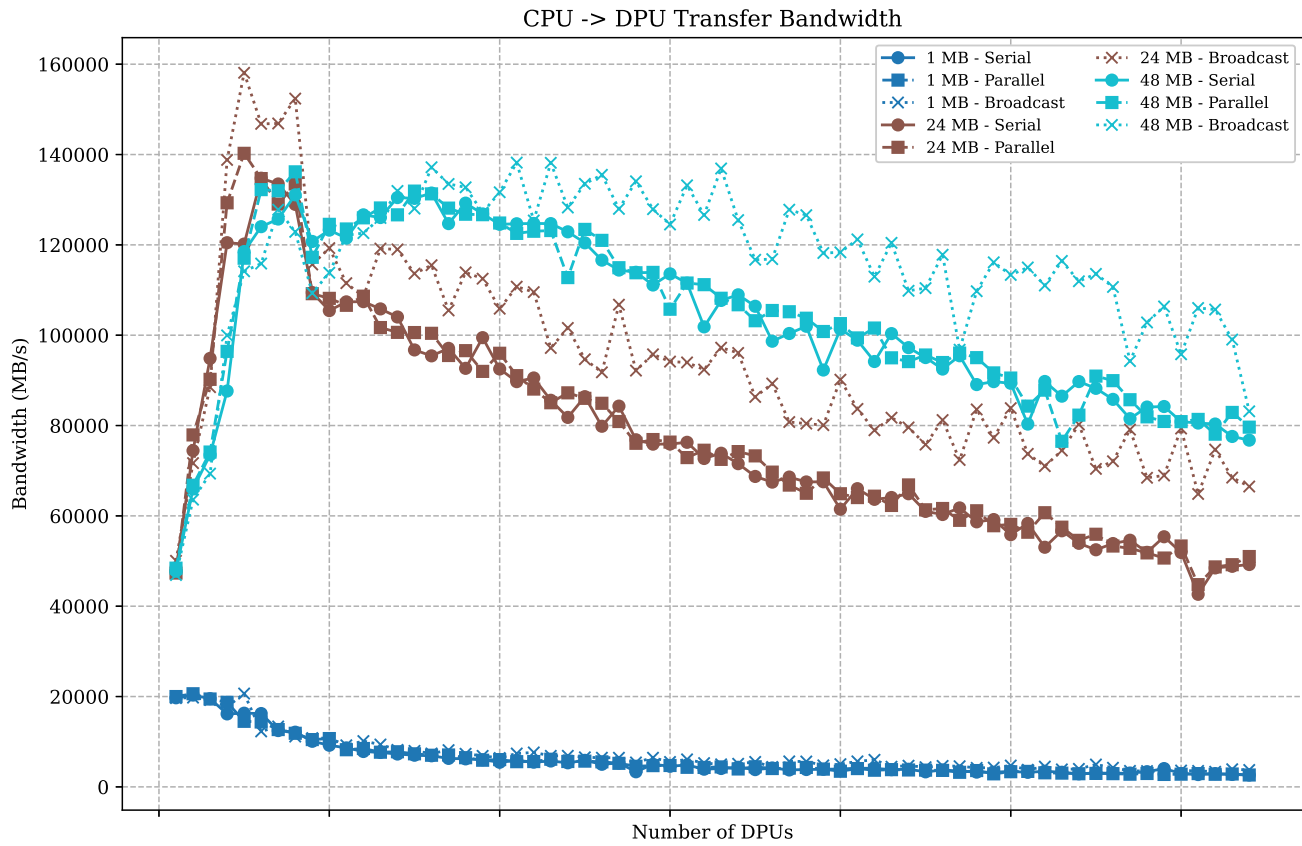


Figure 1: Overall bandwidth vs. number of DPUs for three transfer sizes. Top: CPU→DPU (Serial, Parallel, Broadcast). Bottom: DPU→CPU (Serial, Parallel).

Task 2 AXPY

This task is concerned with quantifying the performance scaling of PIM threads (tasklets). To this end, the AXPY operation ($y = y + \alpha \times x$) is computed in a distributed fashion on the DPUs.

Implementation Details

After placing both input vectors x and y in the DPUs MRAM heap, the host triggers the distributed computation. Each tasklet allocates a section of memory of fixed size in the DPUs WRAM using `mem_alloc`. Then disjoint blocks of the vectors are copied from the MRAM into the previously allocated tasklet WRAM space using `mram_read`. Special care must be taken to ensure this transfer's size is between 8 and 2048 bytes and aligned by 8 bytes. The AXPY operation is performed on the data block in WRAM and subsequently written back to MRAM using `mram_write`. To conclude, the host copies and aggregates the partial results from each DPU.

Due to hardware limitations, the maximum number of tasklets per DPU is 24. The allocated tasklet WRAM block's size has to be chosen such that each tasklet's data can fit inside the WRAM, which becomes an issue at larger numbers of tasklets. It is beneficial to maximize the block size, since this reduces the copying transfer overheads between the MRAM and WRAM. I found the highest possible block size to be 512B, using at most $2 * 24 * 512B = 24 * 576B$ of the 64KB of WRAM, which left enough memory for the remaining variables. This configuration produces correct results for any number of tasklets and DPUs.

Evaluation

In fig. 2, the number of executed instructions per tasklet is plotted against the number of tasklets. For this experiment, two 16MB `uint32_t` input vectors were distributed among 32 DPUs and 64 DPUs. Scaling the number of tasklets reduces the instructions executed on each tasklet, but the relationship is non-linear and exhibits diminishing returns.

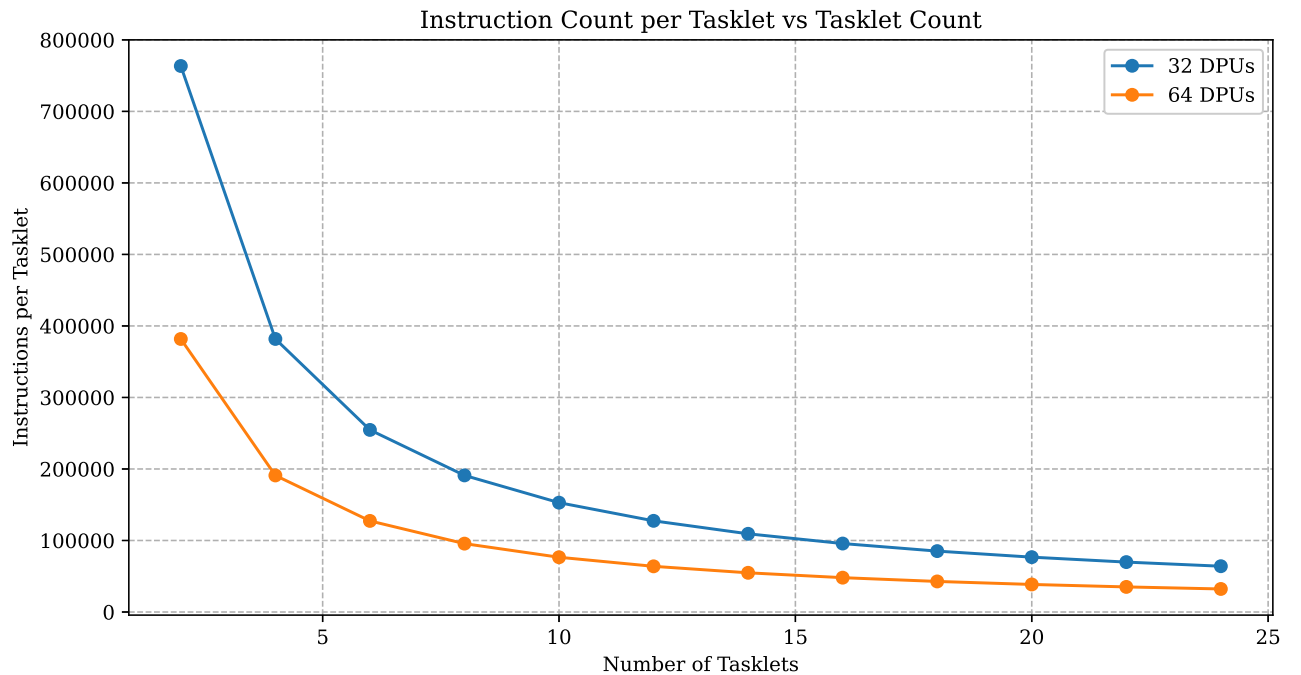


Figure 2: The number of executed instructions per tasklet decreases non-linearly with tasklet count. Using more DPUs (64 vs 32) reduces instructions per tasklet proportionally, as the total workload is distributed across more processing elements.

Analysis and Observations

- As shown in the plot in fig. 2, the number of instructions executed per tasklet decreases as the number of tasklets increases. The curve follows a clear $1/N$ trend. This behavior confirms that the AXPY workload is very parallelizable and that the work is being effectively divided among the available threads.

- The instruction count per tasklet for 64 DPUs consistently is half that of the one with 32 DPUs.
- For the AXPY kernel, adding tasklets beyond around 10 tasklets yields diminishing returns due to fixed overheads and memory transfers. The performance is ultimately likely limited by the bandwidth between WRAM (scratchpad) and the ALU, and the bulk transfers between MRAM and WRAM.

Task 3 Operations and Data Types

The goal of this task is to quantify the computational cost of arithmetic operations on the DPUs across different data types.

Implementation Details

Task 3 builds upon the AXPY kernel from Task 2, but generalizes the computation to support four different arithmetic operations (addition, subtraction, multiplication, and division) across six data types. The operation and data type are selected at compile time via preprocessor macros.

The implementation architecture remains similar to Task 2: input vectors x and y are distributed across DPUs' MRAM heaps with parallel transfers from the host. Each DPU spawns tasklets that collaboratively process their assigned vector chunks.

The core `axpy` function is modified to perform element-wise operations based on the selected macro. Special handling is required for division: for integer types, divide-by-zero is explicitly guarded to prevent SIGFPE signals on the DPUs by checking if the denominator is non-zero before performing the division. For floating-point types, standard IEEE 754 semantics are preserved.

```
// Computes arithmetic operation for a cached block
static void arithm_op(T *bufferY, T *bufferX, unsigned int l_size) {
    for (unsigned int i = 0; i < l_size; i++) {
#ifdef OP_ADD
        bufferY[i] = bufferX[i] + bufferY[i];
#elif defined(OP_SUB)
        bufferY[i] = bufferX[i] - bufferY[i];
#elif defined(OP_MULT)
        bufferY[i] = bufferX[i] * bufferY[i];
#elif defined(OP_DIV)
        /* Protect integer division from divide-by-zero which causes exception */
        if defined(FLOAT) || defined(DOUBLE)
            bufferY[i] = bufferX[i] / bufferY[i];
        else
            bufferY[i] = (bufferY[i] != 0) ? (bufferX[i] / bufferY[i]) : (T)0;
    }
}
```

Transfers between the WRAM and MRAM (`mram_write()`) and the DPU and host (`dpu_push_xfer`) have a minimum transfer size of 8 bytes. For smaller types (CHAR, SHORT, INT32) a larger chunk of 8 bytes is copied and the host extracts the valid bytes using `memcpy()`.

Methodology for Result Generation

To ensure reproducibility and automate the evaluation across the wide range of configurations (operations, data types, and tasklet counts), I utilized a Python script (`profile.py`) to manage the build and execution process. This script iterates through the defined parameter space and, for each unique configuration, sets the corresponding environment variables (e.g., `OP=ADD`, `TYPE=INT32`, `NR_TASKLETS=16`). It then triggers a rebuild of the project by invoking `make`. The slightly modified Makefile captures these environment variables and passes them as preprocessor macros (e.g., `-DOP_ADD`) to the compiler. Once compiled, the script executes the host binary, parses the output for performance metrics (such as instruction counts and execution time), and logs the results to a CSV file, which is subsequently used to generate the plots.

Evaluation

To measure the arithmetic throughput of each combination of data type and arithmetic operation, I measured the number of DPU kernel instructions required to compute 500'000 respective arithmetic operations. The results are plotted in fig. 3.

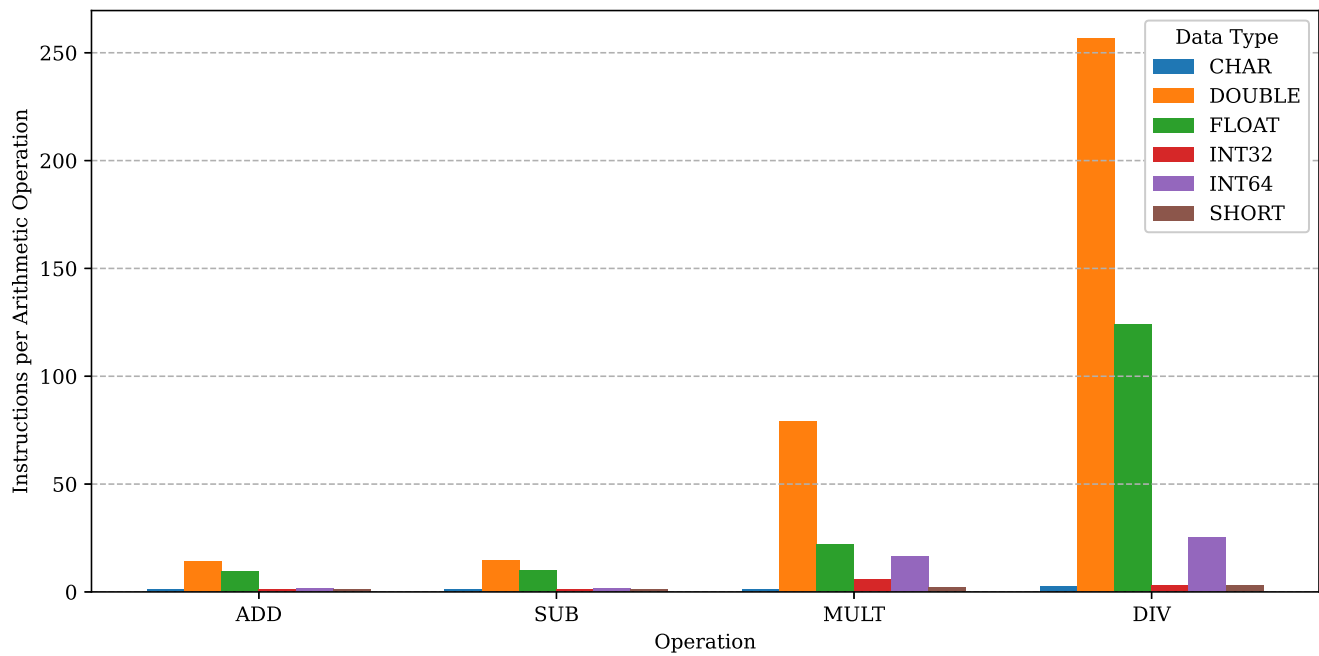


Figure 3: Instructions per arithmetic operation with different operations and datatypes. WRAM block size = 1024B, 16 tasklets, 8 DPUs, 500k operations. Demonstrates the high costs of software emulation.

The results clearly indicate that integer addition and subtraction can be performed with very few instructions, while integer division and multiplication is considerably more expensive. Floating point operations are very expensive, especially for division and multiplication.

Analysis and Observations

DPUs provide native hardware support for 32- and 64-bit integer addition and subtraction, leading to high throughput for these operations. DPUs don't natively support 32- and 64-bit multiplication and division and floating point operations. These operations need to be emulated by the UPMEM runtime, leading to much lower throughput. This is a clear indicator that developers must avoid costly floating point operations, and use integer division and multiplication sparingly.

Task 4 Vector Reduction

This task explores intra-DPU synchronization primitives by implementing parallel vector reduction across multiple tasklets. The goal is to compare four synchronization strategies and identify which approach best leverages the UPMEM architecture’s threading model across different workload sizes.

Implementation Details

For simplicity, my reduction method computes the sum of all vector components. This means that the vector reduction kernel builds on the MRAM and WRAM transfer patterns established in previous tasks but introduces a new final step: after each tasklet independently calculates the sum of its assigned vector partition’s elements, the partial results are merged into an overall sum for the DPU. This final reduction step requires synchronization to prevent race conditions and ensure correctness. The final sum value is computed on the host CPU as the sum of all partial DPU results.

Brief Overview of Implemented Final Reduction Methods

Single Tasklet Reduction (SINGLE) This method performs the reduction serially using a single thread. All tasklets first synchronize at a global barrier to ensure the parallel vector addition is complete. Once synchronized, Tasklet 0 iterates through the `tasklet_partials` array, accumulating the results from all other tasklets into a local variable, while other tasklets remain idle.

Tree Reduction with Barriers (TREE_BARRIER) This method implements a logarithmic tree-based reduction where the number of active tasklets halves at each step (stride 1, 2, 4, etc.). At each level of the tree, active tasklets add a partial result from a “partner” tasklet at `id + stride`. My implementation places a global `barrier_wait(&my_barrier)` at the end of every reduction step. This forces all tasklets (even those that have finished their part of the reduction) to wake up and synchronize at every level of the tree.

Tree Reduction with Handshakes (TREE_HANDSHAKE) Similar to the barrier method, this uses a logarithmic tree structure. However, instead of a global barrier, it uses point-to-point synchronization.

- Receivers (tasklets continuing to the next level) call `handshake_wait_for(partner)` to wait specifically for their partner.
- Senders (tasklets finishing their work at the current level) call `handshake_notify()` to signal they are ready, and then exit the active reduction process. This allows tasklets to synchronize only with their specific partners rather than the entire group.

Mutex-Based Reduction (MUTEX) This method uses a shared global accumulator variable protected by a critical section. Each tasklet independently acquires a lock using `mutex_lock(my_mutex)`, adds its partial result to the global sum, and then releases the lock with `mutex_unlock(my_mutex)`. A final barrier ensures all tasklets have added their values before the result is returned.

Evaluation

The four synchronization strategies were evaluated across three vector sizes (16 KB, 1 MB, 16 MB) distributed among 64 DPUs, with tasklet counts ranging from 4 to 24. For faster runtimes, the underlying data was of the `uint32_t` data-type. Figure fig. 4 shows the total instruction count per DPU as a function of tasklet count for each method at the three vector sizes.

SINGLE, TREE_HANDSHAKE, and MUTEX show nearly identical instruction counts at most configurations. This convergence is surprising given their fundamentally different synchronization models. Across all vector sizes, though especially the smallest one, the TREE_BARRIER reduction incurs higher instruction counts, with the gap widening as tasklet count increases.

Analysis and Observations

Observations on Instruction Count and Scaling

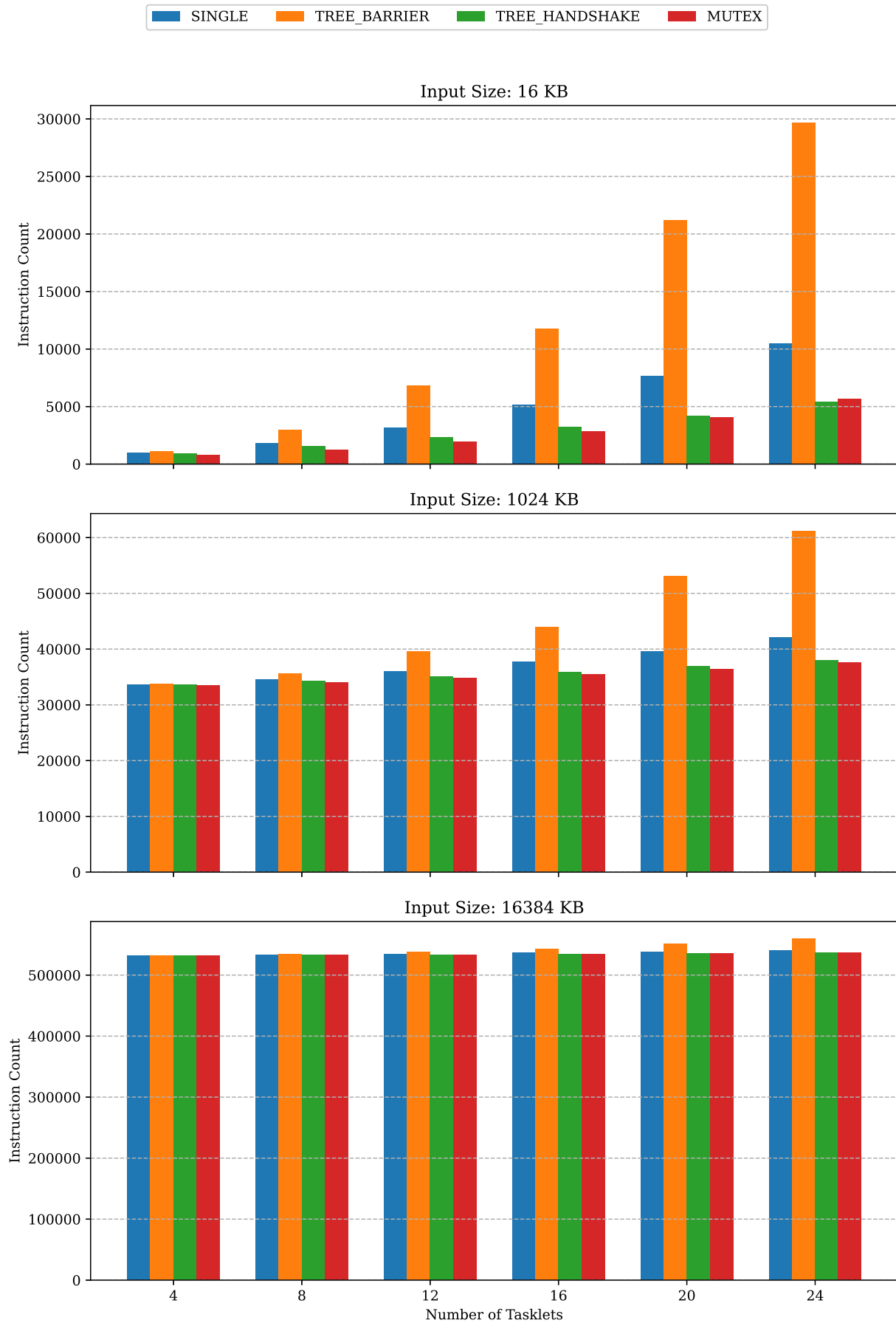


Figure 4: Total instruction count per DPU for four synchronization methods across three vector sizes (16 KB, 1 MB, 16 MB). Particularly at larger tasklet counts the tree barrier method has outsized synchronization costs.

- For small input sizes (16 KB), the execution time is dominated by the reduction logic rather than the vector summation itself.
- For large input sizes (16 MB), the performance difference between reduction methods becomes negligible. The execution is dominated by the $O(N)$ parallel vector addition in the main loop.
- The `TREE_BARRIER` method consistently exhibits the highest instruction count.

Observations of Instruction Count and Scaling

- The `TREE_BARRIER` method forces global synchronization at every step of the reduction tree. Even tasklets that have finished their work (e.g., in early strides) must wake up and participate in the `barrier_wait` for all subsequent stages. This results in $O(N \log N)$ total barrier interactions. In contrast, `TREE_HANDSHAKE` allows inactive tasklets to exit the loop immediately, resulting in significantly fewer executed instructions.
- As the number of tasklets increases to 24, the `MUTEX` and `TREE_BARRIER` methods show steeper increases in overhead compared to `TREE_HANDSHAKE`. `MUTEX` suffers from serialization (contention for the lock), while `TREE_BARRIER` suffers from the increased cost of synchronizing more threads globally.